

Real-World Code Vulnerability Detection Framework: From Data Preprocessing to Multi-Feature Fusion Detection

Jixian Zhang, Qingfeng Du, Sheng Li, Zhongda Lu, Ting He, Chengwei Liu

Abstract—Code vulnerability detection (CVD) is a critical approach to ensuring the security, stability, and reliability of software. When exploited by malicious actors or hackers, code vulnerabilities can lead to a series of severe consequences and cause significant losses. However, the effectiveness of real-world CVD is currently unsatisfactory, with several issues that need to be optimized and resolved. These issues include the poor quality of real-world CVD datasets, the high resource consumption of code intermediate structures, the imbalance between positive and negative samples, and insufficient feature modeling of code vulnerabilities. To address these challenges, we propose a comprehensive and efficient framework for real-world CVD called MARCOVul. It offers a complete process from data preprocessing to final vulnerability detection, optimizing the entire real-world CVD pipeline. Our approach begins with a data derivation technique which seeks to improve the overall quality of the dataset. Next, we propose a code-specific data augmentation method to tackle the issue of sample imbalance in the dataset. We then propose a code intermediate structure simplification method to reduce computational complexity and resource consumption while fully leveraging the power of language models. Finally, we propose a real-world CVD method based on multi-feature fusion to identify potential security vulnerabilities in the code. Experiments on a large-scale real-world CVD dataset demonstrate the effectiveness of MARCOVul in detecting real-world code vulnerabilities, achieving up to 12.75% BF1 and 6.98% MCC improvements over the best unweighted baselines, and 1.55% BF1 and 1.42% MCC gains over the best weighted ones.

Index Terms—Code vulnerability detection, language model, multi-feature fusion, structure simplification, data augmentation.

I. INTRODUCTION

WITH the wide application of technologies such as cloud computing, the Internet of Things, and big data analytics, software systems are becoming increasingly complex and contain more and more vulnerabilities. The Common Weakness Enumeration (CWE) classifies different types of vulnerabilities, such as Cross-Site Scripting (CWE-79), Out-of-Bounds Write (CWE-787), and SQL Injection (CWE-89), which rank in the top three in the CWE Top 25 Most Dangerous Software Weaknesses for 2023 [1] and 2024 [2]. Hackers can exploit these vulnerabilities to launch attacks. At the same time, continuous growth of software size introduces multiple challenges for static code vulnerability detection (CVD),

J. Zhang, Q. Du (Corresponding author), and S. Li are with the School of Computer Science and Technology, Tongji University, Shanghai 201804, China. E-mail: {jixian, du_cloud, li_sheng}@tongji.edu.cn

Z. Lu, T. He, C. Liu are with the Beijing Baolande Software Corporation, Beijing, China. E-mail: {zhongda.lu, ting.he, chengwei.liu}@bessystem.com

```
1 void f() {
2     char arr[10];
3     char *p = arr + 20; // Undefined behaviour, pointer arithmetic 'arr+20' is out of bounds.
4 }
```

Fig. 1. An example of SARD vulnerability function.

```
1 // Vul_0:
2 > static Image *ReadOnePNGImage(MngInfo *mng_info, const ImageInfo *image_info, ExceptionInfo *exception) ...
1113 // Vul_1:
1114 > static Image *ReadOnePNGImage(MngInfo *mng_info, const ImageInfo *image_info, ExceptionInfo *exception) ...
2228 // End
```

Fig. 2. An example of real-world code long vulnerability function and its repair function.

including low reliability and limited adaptability in practical engineering scenarios. These challenges have become critical barriers to the widespread application of this technology.

In response to this situation, some studies leverage machine learning techniques to improve the reliability of CVD, while others develop configurable tools to address vulnerabilities in practical engineering scenarios [3]. Among various programming languages, C/C++ has attracted particular attention due to its widespread use and the potential impact of its vulnerabilities [4]. Our study focuses on using machine learning techniques to statically detect C/C++ function-level vulnerabilities in real-world projects (binary classification). Static analysis does not require the code to be executed and can identify most types of vulnerabilities.

Although research on function-level CVD for C/C++ has made some progress in recent years, the detection performance on real-world code remains unsatisfactory. Most existing studies and datasets focus on synthetic data, while those targeting real-world code are relatively scarce. The most widely used synthetic dataset, the Software Assurance Reference Dataset (SARD), mainly contains simple test cases and fails to adequately reflect the complex vulnerabilities in real-world code. For example, the sample in SARD shown in Figure 1 consists of a short five-line function, while the sample shown in Figure 2 from the GitHub project ImageMagick contains over a thousand lines of code, along with more interfering factors.

Since the functions in synthetic datasets are relatively short, with significant vulnerability features and almost no interference factors, models trained on these datasets can achieve desirable detection results. However, when applied to real-world code, detection performance is often poor. To address this challenge, this study proposes a comprehensive real-world CVD framework called MARCOVul by deeply analyzing the complexity and potential influencing factors in real-world

code, which provides a solution with practical application value for the field of software security.

We analyze the current issues in real-world CVD, which mainly include the following aspects.

1) *Quality of the Dataset*: There are already some datasets composed of real-world code. However, despite these datasets have brought breakthroughs in coverage and scale, they generally suffer from quality issues, with sampling accuracy ranging from 25% to 60% [5]. We categorize these problems into three main types: First, there are two types of labeling errors. Some non-vulnerable functions are incorrectly labeled as vulnerable due to unreasonable annotation rules during the crawling stage, while some vulnerable functions are mislabeled as non-vulnerable because the discovery of vulnerabilities lags behind their introduction into the codebase. [6]. Second, the definition of vulnerabilities is ambiguous. Behavioral differences of the same function in different contexts lead to inconsistent labels, and there is controversy over the vulnerability classification of many functions that contain only return statements or are empty. Finally, the dataset suffers from improper splits. Random splitting fails to ensure that the training set adequately covers vulnerability patterns related to CWE types and projects, which impacts the effectiveness of model training.

2) *Severely Imbalanced Nature of Samples*: In real-world codebases, vulnerable code is extremely rare compared to normal code. Among vast repositories, only a very small percentage of code will be recognized and labeled as containing security vulnerabilities. This distribution characteristic results in severe class imbalance when training with real-world code samples. The imbalance of samples presents a challenge for training machine learning models, as models may tend to predict the majority class.

3) *Improper Code Representation*: Currently, code representations are classified mainly into three types: trees, graphs, and sequences. In real-world CVD, the commonly used representations are trees and graphs (e.g., AST, PDG, CFG), which capture different aspects of code features. However, these representations present several issues. On the one hand, these code representations are very large in real-world datasets, typically containing thousands of nodes, making it extremely challenging to extract useful features from these trees and graphs. On the other hand, most methods currently use only a subset of code representations, leading to insufficient utilization of code representations. In real-world CVD, some vulnerabilities cannot be effectively captured by using tree or graph structures, such as the vulnerability fix of CWE-79:

```

1  rndr_underline(struct buf *ob, const struct buf *text,
2                      void *opaque)
3  {
4      if (!text || !text->size)
5          return 0;
6      /*- BUFPUTSL(ob, "<q>"); */
7      /*+ BUFPUTSL(ob, "<u>"); */
8      bufput(ob, text->data, text->size);
9      /*- BUFPUTSL(ob, "</q>"); */
10     /*+ BUFPUTSL(ob, "</u>"); */
11     return 1;
12 }

```

The fix merely replaces `<q>` with `<u>`, and there is no distinction in tree and graph structures. Especially after

normalization [7], both the tree and graph structures are identical. In this case, only the semantic information of the text sequence can be used for detection.

However, some studies also indicate that when the source code is treated as a single text, the performance of the model in CVD is not ideal [8]. Therefore, since different vulnerabilities require different code representations for detection, using single or partial representations is often insufficient to fully represent the code and detect vulnerabilities. Currently, most methods rely on single or partial representations, leading to inadequate code representation.

To address these challenges, this study aims to design and construct a comprehensive vulnerability detection framework for real-world code. The framework covers the entire process from data preprocessing to final vulnerability detection. The key research contributions of this study are as follows:

- 1) **Derived Dataset Construction**: Based on the CWE type and the Project of the data, we design an algorithm to derive existing datasets, generating a higher-quality dataset. Additionally, the dataset is partitioned based on CWE-Project to make its data distribution more reasonable.
- 2) **Code-Specific Data Augmentation Method**: We propose a code-specific data augmentation technique to balance the training dataset, thus avoiding bias towards categories with larger data during the training process.
- 3) **Code Representation Optimization Methods**: We propose several code representation optimization methods, including intermediate structure simplification and multi-feature fusion. Intermediate structure simplification reduces the overall graph size while retaining the original features to fully leverage the capabilities of language models. Multi-feature fusion combines different code representations, forming a comprehensive code representation for detecting real-world code vulnerabilities.
- 4) **Code Vulnerability Detection Framework**: We design a code vulnerability detection framework called MARCOVul that optimizes the entire process from data preprocessing to final vulnerability detection, providing a solution with practical application value for the software security field.

We have open-sourced all of the code, raw data, intermediate data, and model data used in this study, which can be found on GitHub at: <https://github.com/zjx-forever/MARCOVul>.

II. RELATED WORK

The effectiveness of real-world CVD methods hinges on three key factors: the dataset, code representation, and model architecture. All three are indispensable. Therefore, we introduce them from three perspectives in turn.

A. Dataset

The datasets currently used for static vulnerability detection in C/C++ code are mainly divided into two categories: artificially constructed datasets and real-world datasets.

Synthetic datasets (e.g., SARD [9] and its derivatives) are widely used in model training and effectiveness validation [7], [10]–[20]. However, recent studies reveal a key issue:

models that perform excellently on artificially constructed datasets often fail to effectively address complex challenges in real-world scenarios. Therefore, the current research trend is gradually shifting towards the use of real-world code datasets.

TABLE I
BASIC INFORMATION OF DATASETS

Dataset	Project Num	CWE Num	Num(Vul / No-Vul)
Devign	2	\	12460 / 14858
Reveal	2	\	2240 / 20494
CrossVul	498	107	6,884 / 134,126
CVEFixes	564	127	8,932 / 168,089
BigVul	348	91	11,823 / 264,919
MegaVul	992	169	17,380 / 322,168
DiverseVul	797	150	18,945 / 330,492

Real-world CVD datasets are primarily collected from GitHub, defect repositories (e.g., NVD), and security issue websites (e.g., Snyk, Red Hat), including datasets such as Devign [21], Reveal [22], CrossVul [23], CVEFixes [24], BigVul [25], MegaVul [26], and DiverseVul [5]. The basic information of these datasets is shown in Figure I.

The Devign dataset is collected from the Qemu and FFmpeg projects on GitHub and is a balanced dataset, but the distribution of vulnerabilities differs from the distribution of vulnerabilities in real-world code. Several studies [5], [13], [18], [21], [27]–[30] use the entire Devign dataset for model training and validation, while some studies [19] only use data from the Qemu project. The Reveal dataset is collected from the Chromium and Debian projects on GitHub and is an imbalanced dataset. Our preliminary research indicates that, due to its imbalance, the detection performance on Reveal is significantly lower than on Devign. Both of these two datasets do not provide CWE-related information.

Except for these two datasets, all the other datasets are imbalanced and include data from multiple projects as well as various CWE labels. Among them, the DiverseVul dataset has the largest amount of data, while the MegaVul dataset offers the greatest diversity in terms of projects and CWE types.

These datasets differ in terms of field composition, organizational structure, and information richness. To facilitate data selection and processing, we define a set of key fields (*func*, *target*, *cwe*, *project*) and establish a standardized format. We then invite five scholars to assess the level of standardization of the raw data in each dataset. The results indicate that CrossVul and CVEFixes are raw data with minimal data cleaning, so they are classified as *Low*. The MegaVul is relatively standardized, but contains a large amount of redundant information beyond the key fields, so it is classified as *Middle*. DiverseVul, BigVul, Devign, and Reveal follow standardized data formats and are classified as *High*. Based on this assessment, we select the experimental dataset for this study.

B. Code Representation

Code can be represented in many different ways. In static CVD for C/C++ code, the main representations typically fall into three categories: text sequences, trees, and graphs.

1) *Text sequences*: The most direct representation of code is text sequences, typically consisting of a series of tokens. By applying embedding operations on the code text, vector representations containing the semantic features of the code can be obtained.

2) *Tree*: The representative of the tree representation type is the Abstract Syntax Tree (AST). The AST is an abstract representation of source code, expressed in a tree-like structure that abstracts the syntactic structure of a program. This abstraction makes the AST an ideal intermediate representation for many stages in a compiler and one of the most widely used representations of code. In previous research, the AST has been widely adopted as a code representation method. Study [17] focus on using a single AST structure to identify security vulnerabilities in code.

3) *Graph*: The types of graph representation are diverse and can capture code characteristics from different dimensions.

Control Flow Graph (CFG) is a graphical representation used to describe the control flow within a program. It is a directed graph where the nodes represent basic blocks and the edges represent control flow paths. A variant of the CFG is the Interprocedural Control Flow Graph [31]–[33], allows models to understand the control flow across the entire program.

Program Dependence Graph (PDG) is a graphical representation used to express the control and data dependencies within a program. It is a directed graph where the nodes represent program statements, and the edges denote dependencies between those statements. PDG contains two types of dependencies and can be viewed as a combination of the Control Dependence Graph (CDG) and the Data Dependence Graph (DDG). SlicedLocator [14] focuses on using PDG slices for CVD. A variant of the PDG is the Program Slices [7], which is a subgraph of the PDG and includes only the nodes relevant to a specific point (node of interest) in the program.

Code Property Graph (CPG) is a unified data structure that combines elements of the AST, CFG, and PDG, making it one of the most comprehensive graph representations of code information. The AST describes the syntactic structure, the CFG represents the execution paths and control flow transitions, and the PDG provides information on data dependencies and control dependencies within the program. VulSniper [10] uses CPG as a code representation, combined with an attention mechanism, to detect vulnerabilities within code. A variant of the CPG is the Simplified Code Property Graph [34], [35], which uses only the AST and CFG edges contained in CPG.

The above three different representations respectively represent the semantic information (text sequences), syntactic structure (AST) and semantic structure (graphs) of the code. Their usage frequency in research papers, in descending order, is as follows: Text sequences, AST, CFG, PDG and CPG [4].

Recent research tends to adopt diversified code representation methods, as a single representation often fails to fully capture the complex characteristics of code functions [15], [16], [18], [19], [27], [36]–[38]. These researches demonstrate that by merging various code representation, researchers can more effectively capture the structural and semantic features of code, providing a solid foundation for CVD.

C. Model Architecture

CVD models are mainly divided into two categories: one type is embedding models, which are responsible for converting code into embedded representations, and the other type is analysis models, which extract features from these embeddings and perform classification.

In previous research, Word2Vec has been a widely used embedding model [12], [16], [18], [19], [39]–[43]. In addition to Word2Vec, other embedding methods such as Doc2Vec [7] and Node2Vec [18], [36] have also been applied to capture code semantics and generate embeddings for CVD.

In recent years, with the rapid development of natural language processing technology, more and more studies have begun to directly utilize language models (LMs) to process code [5], [44]. Some studies [27], [45] use pre-trained models such as CodeBERT [46] to generate code embeddings or directly perform CVD. Furthermore, with the rapid development of large language models (LLMs), an increasing number of pre-trained models available for code have emerged, such as stable-code-3b [47], CodeLlama-70b-hf [48], FlagEmbedding [49], the CodeT5 series [50], [51], and models pre-trained for specific programming languages like C and C++, such as codebert-c and codebert-cpp [52].

After converting code into embedded representations, relevant machine learning models are typically used to capture deeper and more effective features. Different code representation methods often require different machine learning models to effectively extract features.

Some studies use RNN-based models to capture vulnerability features. For example: VulDeePecker [41] and MSGVUL [18] apply Bi-LSTM to extract features. SySeVR [53] uses Bi-GRU for feature extraction. SlicedLocator [14] employs the LSTM-GNN model to capture vulnerability features.

Some studies utilize Transformer models to capture vulnerability features. For example, TrVD [17] uses a tree-structured neural network to capture the semantics of each AST subtree and then uses a Transformer encoder to aggregate the contextual semantics of all subtrees.

Some studies use Gated Graph Neural Networks (GGNNs) to capture vulnerability features. For example: Devign [21] uses Gated Graph Recurrent Layers combined with convolutional layers to extract features. VDoTR [16] uses CircleG-GNN to capture hidden embedded features of vulnerabilities.

In addition, most studies use GNN-based models to capture vulnerability features. For example: Swarna [37] and Vul-Sniper [10] employ attention-based neural networks to capture vulnerability features. CPVD [13] extracts graph feature vectors using Graph Attention Networks (GAT) and convolutional pooling networks, followed by domain-adaptive representation learning for vulnerability detection. DeepWukong [7] uses related GNN models (GCN, GAT, k-GNN) to extract features. CSGVD [27] uses GNN to extract structural information from the graph and applies Mean Bilateral Fusion Attention Pooling to aggregate node information. PFHE-MVD [19] uses Hyperbolic Graph Convolutional Neural Networks to model the hierarchical call structures within the graph.

Moreover, some researchers propose their own models for vulnerability detection. For example, VDTriplet [43] uses a

TripletNN model to detect vulnerabilities and reduce classification errors related to potentially known vulnerabilities.

III. METHOD

A. Overall Framework

To address current issues in real-world CVD, we propose the MARCOVul framework. Figure 3 illustrates the overall architecture of MARCOVul.

The framework consists of five core steps. First, to address the quality issues of the dataset, we design relevant rules and algorithms to derive and partition the dataset, constructing a higher-quality dataset. Then, we use text sequences, tree structures, and graph structures to comprehensively represent the code, generating intermediate structures, respectively. We also propose an effective graph simplification algorithm to simplify the tree and graph representations. This algorithm reduces the number of nodes, lowering computational complexity while fully leveraging the power of language models. Additionally, to solve the class imbalance problem in the training set, we introduce a data augmentation method specifically designed for code data. This method effectively balances the training set, improving the model's ability to recognize minority classes. In the code representation learning module, we embed the above different types of code representations using different models to get the corresponding embedded representations. These four embeddings comprehensively capture the multidimensional features of the code, providing strong support for subsequent classification. Finally, in the classification module, we use the learned embeddings, combine them with an attention mechanism for multi-feature fusion, and apply a classification model to determine whether vulnerabilities exist in the code, achieving efficient and accurate vulnerability detection.

Overall, MARCOVul optimizes the entire process from data preprocessing to final vulnerability detection. The following sections describe the implementation details of each step.

B. Derived Dataset Construction

To improve the quality and usability of datasets, we design a data derivation method consisting of two core steps: data cleaning and data partitioning.

1) *Data Cleaning*: In this research, we focus on analyzing four key fields: *func*, *target*, *cwe*, and *project*. Among these, the values of the *func*, *target*, and *project* fields are unique, while the *cwe* field may contain multiple entries. This is because a single Common Vulnerabilities and Exposures (CVE) entry in the National Vulnerability Database (NVD) may be associated with multiple CWE entries. In most datasets, the labeling method for the *target* field follows these guidelines: if the original version of the code contains a vulnerability, it is labeled as vulnerable(1); the corresponding fixed version is labeled as non-vulnerable(0). For unmodified code, they are uniformly labeled as non-vulnerable(0). Furthermore, those functions identified as 1 and some marked as 0 are associated with the corresponding cwe identifiers, indicating their relationship to specific vulnerability categories.

Based on the characteristics of the datasets, the data cleaning process that we design is divided into the following steps:

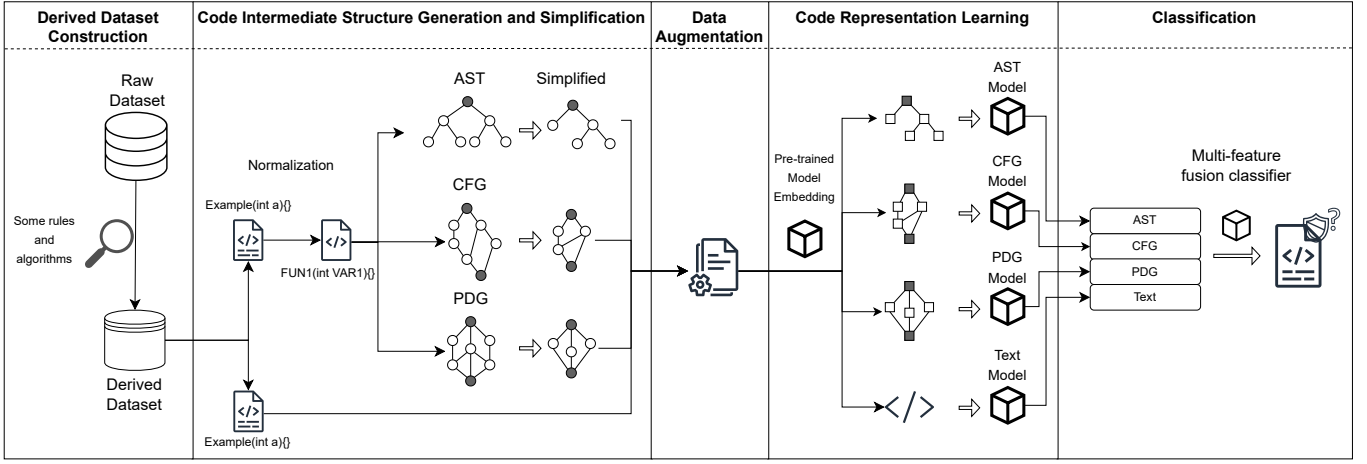


Fig. 3. The overall architecture of MARCOVul. This architecture consists of five main steps: dataset derivation, generation and simplification of code intermediate structures, data augmentation, feature extraction from four different code representations using diverse models, and multi-feature fusion classification based on an attention mechanism.

- (1) **Standardizing Data Format:** We first remove invalid functions that cannot generate CPG (e.g., incomplete or syntactically incorrect functions). Then, we format the data based on key fields and assign a unique identifier (idx) to each record for subsequent processing.
- (2) **Removing Duplicate Data:** We remove duplicate records with the same 'func' value to avoid affecting the analysis.
- (3) **Removing Data with Missing CWE Identifiers:** Some data entries lack CWE identifiers, making it impossible to identify the corresponding vulnerability type and verify whether the vulnerability truly exists. So we remove the entries where the CWE field is empty.
- (4) **Removing Functions with Empty Bodies or Only a Return Statement:** Some functions in the dataset have empty bodies or contain only a return statement. These functions do not provide sufficient vulnerability feature information, and it is unreasonable to label them as either vulnerable or non-vulnerable. Therefore, they are not valid data samples. Based on this, we use regular expressions to identify and remove these functions, ensuring that the remaining functions in the dataset provide valuable information for real-world CVD.
- (5) **Removing Functions Based on Rules:** We design two rule-based filtering strategies to remove unreasonable data, including the following core rules:

Minimum Intra-Class Sample Size Filtering: For each CWE-Project category, if the number of either vulnerable or non-vulnerable functions is less than a predefined threshold (default: 1), the entire category is discarded. This threshold is set based on the rationale that categories with insufficient samples (containing only one class) provide no meaningful discriminative information and are thus not valuable for model training.

Non-vulnerable Function Filtering: For each CWE-Project category, we compute the ratio of non-vulnerable to vulnerable functions. Filtering is performed only when this ratio exceeds a specified threshold (default: 3). The purpose of this filtering is to remove less informative

non-vulnerable functions in order to control the class imbalance in the dataset and to help the model learn deeper vulnerability-related features. However, this process may increase the difficulty of classification. The threshold is chosen to reflect the natural imbalance in real-world data while preserving as many vulnerable functions and their related samples (e.g., patched or similar functions) as possible, avoiding excessive removal.

During the filtering process, if a function is associated with multiple CWE types, we remove only the specific CWE that meets the filtering criteria, while retaining the others. The specific algorithm is shown in Algorithm 1.

In Algorithm 1, we use Levenshtein distance to calculate the similarity between two functions and remove some non-vulnerable functions with low similarity. The Levenshtein distance refers to the minimum number of edit operations required to transform one string into another, including substitution, insertion, and deletion. The formula for calculating the Levenshtein distance is shown in Equation 1:

$$L(i, j) = \begin{cases} \max(i, j) & \text{if } \min(i, j) = 0, \\ \min \begin{cases} L(i-1, j) + 1 \\ L(i, j-1) + 1 \\ L(i-1, j-1) + 1_{(S1_i \neq S2_j)} \end{cases} & \text{otherwise.} \end{cases} \quad (1)$$

where the notation $L(i, j)$ represents $lev_{S1, S2}(i, j)$, which denotes the levenshtein distance between the first i characters of $S1$ and the first j characters of $S2$. $S1$ and $S2$ are two functions, i and j denote the subscripts of $S1$ and $S2$ respectively, with the subscripts starting from 1. The $1_{(S1_i \neq S2_j)}$ is an indicator function that has a value of 0 when $S1_i = S2_j$, and 1 at other times.

The similarity calculation formula based on the Levenshtein distance is given by Equation 2:

$$\text{Similarity} = 1 - \frac{L(|S1|, |S2|)}{\max(|S1|, |S2|)} \quad (2)$$

where $|S1|$, $|S2|$ represent the lengths of $S1$ and $S2$.

Algorithm 1 Removing Functions based on Rules.

Input: *all_data*
Output: *new_all_data*

```

/* Smallest number and Maximum ratio of 0 and 1 in the same cwe-project.*/
1: cnt_min01  $\leftarrow$  1; max_rat_0_1  $\leftarrow$  3;
/* Divide all_data by CWE-Project to form a dictionary with key:cwe-project and value:data-list. */
2: cwe_project  $\leftarrow$  DivideAllDataFormDict(all_data);
/* idx-cwes dictionary: Records cwes to be deleted for a specific idx. */
3: del_idx_cwe  $\leftarrow$  {}
4: for key, values in cwe_project do
5:   cwe_type  $\leftarrow$  The cwe contained in the key.
/* Calculates the number of 0 and 1 in values. */
6:   cnt0, cnt1  $\leftarrow$  Count01InValues(values);
7:   if cnt0 < cnt_min01 or cnt1 < cnt_min01 then
8:     for item in values do
9:       del_idx_cwe[item.idx].add(cwe_type)
10:    end for
11:   else
12:     if cnt0/cnt1 > max_rat_0_1 then
13:       del_num  $\leftarrow$  cnt0 - cnt1 * max_rat_0_1
/* Records all functions in values with target 1*/
14:       target_1_func_list  $\leftarrow$  Rec1InValues(values);
/* Use the Levenshtein distance for each func in values with target 0 to calculate its similarity to all functions in target_1_func_list, recording the greatest similarity. */
15:       sim_list  $\leftarrow$  CalcGTSim0ToAll1(values);
/* Record the del_num idxs with the least similarity in sim_list. */
16:       del_idx  $\leftarrow$  RecDelIdx(sim_list);
17:       for idx in del_idx do
18:         del_idx_cwe[idx].add(cwe_type)
19:       end for
20:     end if
21:   end if
22: end for
23: new_all_data  $\leftarrow$  []
24: for item in all_data do
25:   if item.idx not in del_idx_cwe then
26:     new_all_data.add(item)
27:   else
/* Calculate the remaining cwes after deleting the cwes that need to be deleted, and update item. */
28:     new_item_cwes  $\leftarrow$  CalRemCWES(item, del_idx_cwe);
29:     if len(new_item_cwes)  $\neq$  0 then
30:       new_all_data.add(item)
31:     end if
32:   end if
33: end for
34: return new_all_data

```

We observe that vulnerable functions exhibit high similarity to their patched versions, while unrelated functions show low similarity. The issue is that many unrelated functions labeled as non-vulnerable may actually contain (undiscovered) vulnerabilities due to the lag in vulnerability discovery. Models can easily distinguish vulnerable functions from these low-similarity non-vulnerable ones, but this often relies on superficial features rather than genuine vulnerability patterns. Although retaining these low-similarity functions may temporarily improve performance on the dataset, it leads the model to learn shallow features that do not generalize to real-world code. Therefore, to optimize the quality of the dataset and improve the vulnerability detection ability of the downstream model in real-world code, we remove some functions that are low similar to the vulnerable functions.

2) *Data Partitioning*: To enable the model to distinguish between vulnerable and non-vulnerable functions and capture the features of vulnerabilities, it is crucial that the training set

Fig. 4. An example of code function normalization.

contains both vulnerable and non-vulnerable functions within the same CWE-project. The previous data cleaning operations laid the foundation for data partitioning.

Since a single data entry may involve multiple CWEs, an algorithm is applied during the data partitioning process. This algorithm first identifies which cwe-project among the multiple CWEs involved in the data has the fewest vulnerable and non-vulnerable functions in the training, validation, and testing sets, and carries out subsequent addition operations with the fewest cwe-project. This approach ensures that a relatively small number of cwe can obtain a more reasonable data distribution. When adding data to each set, we split the data in an 8: 1: 1 ratio, with a priority for the training set, followed by the validation set, and lastly the testing set.

C. Code Intermediate Structure Generation and Simplification

In this work, we use the Joern¹ tool to generate intermediate structures (AST, CFG, and PDG) of the code. Joern is a platform designed for robust analysis of source code, bytecode, and binary code, utilizing the CPG as its foundational structure for deep analysis and detection.

In the process of generating intermediate structures of code using the Joern tool, we noticed that individual differences in programming habits can lead to diversity in the semantic features of the code, which are often unrelated to the structural features of the code. Since intermediate structures primarily capture the structured features of the code rather than its semantic features, the diversity in function names and variable names may negatively impact the accurate extraction of vulnerability features. Therefore, we first normalize the code to minimize such interference. An example of normalization is shown in Figure 4. On the other hand, sometimes the original semantics of the code is beneficial to capture vulnerabilities. To preserve the original semantics of the code, we do not normalize its non-structural features. Overall, we first normalize the raw code text before generating the intermediate structures, which are later used to extract structural features. At the same time, we retain the unnormalized raw code text for extracting semantic features from the code.

After generating the intermediate structures, we simplify these structures to reduce computational complexity and fully leverage the processing power of LMs. Through empirical analysis, we find that real-world functions can be extremely complex, with node counts reaching tens of thousands, which presents significant challenges for the model in identifying vulnerability-related features. By observing the intermediate

¹<https://joern.io/>

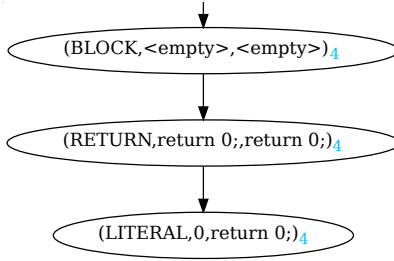


Fig. 5. An example of intermediate structure redundancy in code.

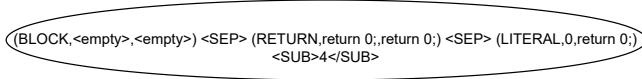


Fig. 6. The result of simplifying the redundant structure in Figure 5.

structure, we find that many nodes are redundant for real-world CVD and can be merged, such as the AST of the statement `return 0;` in the example of Figure 4, as shown in Figure 5. To address this challenge, we propose an effective method for reducing the number of graph nodes based on the graph structure generated by Joern.

When building the graph, Joern assigns a $\langle SUB \rangle x \langle /SUB \rangle$ attribute to each node, indicating which nodes belong to the same subblock (line level). These subblocks could be a single statement or a code block, with x being the unique identifier for each subblock. We utilize this attribute to merge nodes with the same $\langle SUB \rangle x \langle /SUB \rangle$ value into a single supernode. This supernode encapsulates all the information of the original nodes, with different pieces of information separated by $\langle SEP \rangle$, as shown in Figure 6. This approach not only reduces the total number of nodes, but also preserves the original information of the nodes, which is crucial for subsequent vulnerability detection. For functions with a simpler structure that contain only a single subblock (i.e., the generated $\langle SUB \rangle x \langle /SUB \rangle$ attribute has only one category), the number of nodes is already relatively low, so we do not perform node reduction on these functions. This method ensures that, while reducing overall computational complexity, we still retain the key details of smaller functions.

Through this method, we effectively handled large functions while maintaining a deep understanding of the code structure.

D. Data Augmentation

After simplifying the intermediate structures, we propose a data augmentation method to increase the number of vulnerable samples, aiming to balance the sample ratio in the training set. This method consists of three steps:

1) *Calculating the augmentation factor*: We group the training set by CWE-Project and determine the augmentation factor for each group based on the ratio of non-vulnerable to vulnerable samples. The factor is calculated by rounding down to ensure that the number of vulnerable samples does not exceed the number of non-vulnerable samples. During data augmentation, the vulnerable functions in each group are augmented based on the group-specific augmentation factor.

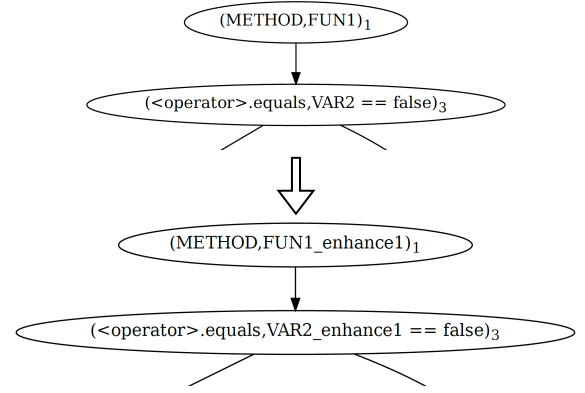


Fig. 7. The result of data augmentation applied to part of the CFG structure from the example in Figure 4.

2) *Augmenting the code intermediate structures*: We adopt an effective method to augment code intermediate structures without altering their structural characteristics. Before generating the intermediate structures, we normalize the data to eliminate the semantic influence of function names and variable names, where all variables and function names are represented as $VAR\{x\}$ and $FUN\{x\}$. This uniform representation ensures that structural feature analysis is not affected by naming conventions. Based on this, our data augmentation method involves replacing variable names $VAR\{x\}$ and function names $FUN\{x\}$ in the intermediate structure by appending a suffix $_enhance\{i\}$, where i is the sequence number of enhanced data, as shown in Figure 7. This approach not only increases the diversity of the data, but also reduces the risk of overfitting that the model might encounter during training by introducing perturbations. In this way, our model can better adapt to different naming styles and coding conventions, enhancing its robustness in real-world applications.

3) *Augmenting the code text sequence*: Since code text sequences are designed to capture semantic information, our aim is to increase the diversity of the dataset without altering the original code semantics. We adopt a subtle perturbation method that only targets function names. Specifically, we append an augmentation suffix $_enhance\{i\}$ to the function name identifiers, where i represents the augmentation sequence number. This method preserves the code's semantics while also enriching the training data to some extent.

E. Code Representation Learning Module

To capture the features of code more comprehensively, we extract effective code features related to vulnerabilities from the code text sequence, AST, CFG, and PDG. The code text sequence is used to capture semantic information, the AST is used to capture syntactic structure, and the CFG and PDG are used to capture semantic structure and reveal the execution paths and dependencies of the program. We assume that different code representation methods require different machine learning models for effective feature extraction, so each part uses a specific model to extract features. The detail of this module is shown in Figure 8.

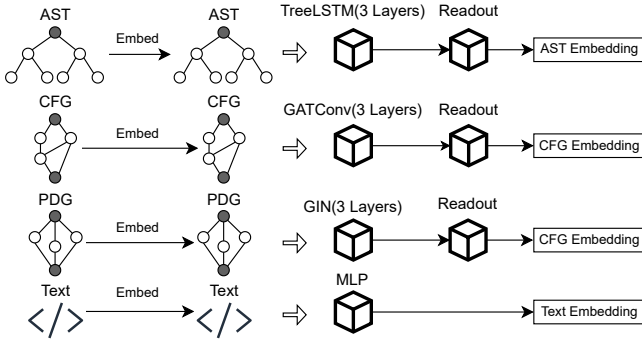


Fig. 8. The details of the module for using different models to perform feature extraction on different code representations.

The input to this module includes the simplified and data-augmented AST, CFG, PDG, and the data-augmented code text sequence. When processing AST, CFG, and PDG, we conduct preliminary experiments, which indicate that using pre-trained language models for embedding is better than using Word2Vec+RNN. Therefore, we first utilize a fine-tuned pre-trained language model on the training set to embed node content, transforming nodes into vector representations.

Subsequently, we use different models to capture the multi-dimensional features of the code's tree and graph structures.

For AST, because it is a hierarchical tree structure with a variable number of child nodes, we adopt the Child-Sum Tree-LSTM model, which is specifically designed for tree-structured data and can effectively handle nodes with varying numbers of children. Before capturing the hierarchical feature, we precompute the node and edge ordering. The calculation formula of Child-Sum Tree-LSTM [54] is as follows:

$$\begin{aligned}
 \tilde{\mathbf{h}}_j &= \sum_{k \in C(j)} \mathbf{h}_k \\
 \mathbf{i}_j &= \sigma \left(\mathbf{W}^{(i)} \mathbf{x}_j + \mathbf{U}^{(i)} \tilde{\mathbf{h}}_j + \mathbf{b}^{(i)} \right) \\
 \mathbf{f}_{jk} &= \sigma \left(\mathbf{W}^{(f)} \mathbf{x}_j + \mathbf{U}^{(f)} \mathbf{h}_k + \mathbf{b}^{(f)} \right) \\
 \mathbf{o}_j &= \sigma \left(\mathbf{W}^{(o)} \mathbf{x}_j + \mathbf{U}^{(o)} \tilde{\mathbf{h}}_j + \mathbf{b}^{(o)} \right) \\
 \mathbf{u}_j &= \tanh \left(\mathbf{W}^{(u)} \mathbf{x}_j + \mathbf{U}^{(u)} \tilde{\mathbf{h}}_j + \mathbf{b}^{(u)} \right) \\
 \mathbf{c}_j &= \mathbf{i}_j \odot \mathbf{u}_j + \sum_{k \in C(j)} \mathbf{f}_{jk} \odot \mathbf{c}_k \\
 \mathbf{h}_j &= \mathbf{o}_j \odot \tanh(\mathbf{c}_j)
 \end{aligned} \tag{3}$$

where $C(j)$ represents all child nodes of node v_j , $\mathbf{i}_j$ is the input gate, $\mathbf{f}_{jk}$ is the forget gate of node v_j and child node v_k , $\mathbf{o}_j$ is the output gate, $\mathbf{u}_j$ is the candidate cell state, $\mathbf{c}_j$ and $\mathbf{h}_j$ represent the updated cell state and hidden state of node v_j , respectively. σ represents the Sigmoid activation function.

For CFG, since it is a directed graph representing the control flow, we use GAT, which is suitable for capturing transfer characteristics, to extract vulnerability features, using the Leaky_ReLU activation function. Additionally, to retain critical information while reducing graph size, we apply Self-

Attention Graph Pooling (SAGPooling) after each layer of GAT. The calculation formula of GAT [55] is as follows:

$$\begin{aligned}
 \mathbf{e}_{ij} &= \text{LeakyReLU}(\mathbf{W}^T [\mathbf{W} \mathbf{h}_i \parallel \mathbf{W} \mathbf{h}_j]) \\
 \alpha_{ij} &= \text{softmax}_j(\mathbf{e}_{ij}) = \frac{\exp(\mathbf{e}_{ij})}{\sum_{v_k \in N(v_i)} \exp(\mathbf{e}_{ik})} \\
 \mathbf{h}'_i &= \sigma \left(\sum_{v_j \in N(v_i)} \alpha_{ij} \mathbf{W} \mathbf{h}_j \right)
 \end{aligned} \tag{4}$$

where v_i is the central node, $N(v_i)$ is the set of neighbor nodes of node v_i , ' $\parallel$ ' denotes the concatenation of feature vectors, $\mathbf{e}_{ij}$ is the attention weight from the neighboring node v_j to v_i , α_{ij} is the normalized weight, and $\mathbf{h}'_i$ represents the updated feature vector of node v_i .

For PDG, considering that it is a directed dependency graph that incorporates both CDG and DDG edges and is typically large in size, we employ the Graph Isomorphism Network (GIN) to capture its complex features. GIN is well-suited for distinguishing graph structures and offers relatively low memory overhead, making it appropriate for capturing the complex dependency features of PDG. The calculation formula of GIN [56] is as follows:

$$\mathbf{h}_{v_i}^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) \cdot \mathbf{h}_{v_i}^{(k-1)} + \sum_{v_j \in N(v_i)} \mathbf{h}_{v_j}^{(k-1)} \right) \tag{5}$$

where $N(v_i)$ is the set of neighbor nodes of node v_i and ϵ is a learnable parameter or a fixed parameter.

The Tree-LSTM, GAT, and GIN models are all configured with three layers. This design choice is based on the findings of existing studies (e.g., LightGCN [57]) as well as our experiments, which indicate that a three-layer architecture is sufficient to achieve satisfactory performance. Increasing the depth beyond three layers yields limited gains and may even lead to performance degradation.

After each step, we use a graph readout layer based on a global attention mechanism to obtain the graph-level embeddings for AST, CFG, and PDG. For the code text sequences, we directly use a pre-trained language model that has been fine-tuned on the training set to extract the semantic embedding of the code, followed by an MLP layer.

After the code feature learning module, we obtain the vector representations corresponding to the four types of features, which are then input into a classifier for classification.

F. Classification Module

In order to combine the graph-level embeddings of the AST, CFG, and PDG with the semantic embedding of the code, we use a multi-feature fusion method based on the attention mechanism. The method forms a comprehensive code representation that can be used to classify whether the code contains vulnerabilities. The detail of this method is shown in Figure 9. Where $\mathbf{e}_{\text{AST}}$, $\mathbf{e}_{\text{CFG}}$, $\mathbf{e}_{\text{PDG}}$, and $\mathbf{e}_{\text{Text}}$ represent AST embedding, CFG embedding, PDG embedding, and Text embedding, respectively. FC represents the fully connected

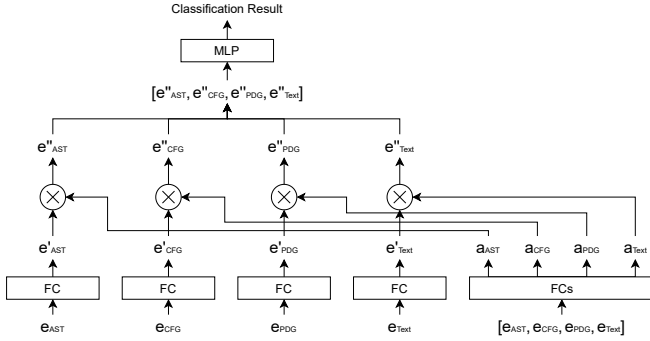


Fig. 9. Details of Multi-feature Fusion Based on Attention Mechanism.

layer. Let $\Gamma = \{AST, CFG, PDG, Text\}$ represents the set of code representations used.

Based on the code feature vectors $e_{AST}, e_{CFG}, e_{PDG}$ and e_{Text} obtained from the previous module, we first calculate the vector e' , which can be expressed by the following formula:

$$e'_t = LeakyReLU(BatchNorm(FC(e_t))), t \in \Gamma \quad (6)$$

Then we calculate the attention score a_t for each type through the attention layer as:

$$a_t = \sigma(FC([e_{AST}, e_{CFG}, e_{PDG}, e_{Text}])), t \in \Gamma \quad (7)$$

Afterwards, we use the computed a'_t and e'_t to calculate the final comprehensive code representation embedding. The formula is as follows:

$$e_{final} = \bigoplus (a_t * e'_t), t \in \Gamma \quad (8)$$

where $\bigoplus$ indicates the horizontal stack.

By integrating these rich code representations, we are able to obtain a comprehensive code embedding. Finally, this code embedding is fed into the MLP for the final classification decision. This integrated representation approach enables our model to understand the code from multiple perspectives, thereby improving the effect of real-world CVD.

IV. EXPERIMENTS

A. Research Question

Our experiments aim to answer the following four research questions (RQs):

- RQ1: Does dataset derivation lead to the generation of higher-quality datasets?
- RQ2: Can intermediate structure simplification and data augmentation improve the effectiveness of real-world CVD?
- RQ3: Can MARCOVul effectively improve the performance of real-world CVD?
- RQ4: Is it effective to capture different code representations using different models?

B. Experimental Setup

1) *Experimental Environment*: All experiments in this work are conducted on a server equipped with an Intel(R) Xeon(R) Gold 6226R CPU @ 2.90GHz (64 processors), 128 GB memory, and an NVIDIA GeForce RTX 4090 24G GPU. The specific experimental configurations and evaluation settings for each experiment are introduced in the corresponding sections.

2) *Dataset*: The previous study [5] evaluates the label accuracy of four datasets: DiverseVul, CVEFixes, CrossVul, and BigVul. Their accuracy rates are 60%, 51.7%, 47.8%, and 25%, respectively, and DiverseVul achieves the highest accuracy. Furthermore, based on our prior evaluation, the standardization rating of DiverseVul is *High*. Therefore, we chose to use DiverseVul for further data derivation work. The actual dataset version we obtained is diversevul_20230702, which contains 800 projects and 150 types of CWE, with 18,945 vulnerable samples and 311,547 non-vulnerable samples.

After data derivation, we obtained the primary dataset used in this study, DiverseVul-Derived, which consists of 577 projects and 135 types of CWE, with 10,886 vulnerable samples and 33,460 non-vulnerable samples. We have made this dataset publicly available on Zenodo: <https://doi.org/10.5281/zenodo.14878568>.

In all datasets, positive samples (1) indicate vulnerabilities, while negative samples (0) represent non-vulnerabilities.

To trace the data derivation process, we record the number of remaining samples, CWE types, and projects after each derivation step. Detailed statistics are presented in Table II.

TABLE II
DATASET SIZE AND DISTRIBUTION OF CWES & PROJECTS DURING DERIVATION.

Dataset	Vul	No-Vul	Total	CWE	Project
Original ¹	18945	311547	330492	150	800
Step 1 ²	14781	249032	263813	147	766
Step 2 ³	14781	249032	263813	147	766
Step 3 ⁴	12547	212461	225008	147	712
Step 4 ⁵	10922	182054	192976	144	697
Step 5 ⁶	10886	33460	44346	135	577

¹ Original: The original dataset that has not undergone data derivation.

² Step 1: The dataset after the 'Standardizing Data Format'.

³ Step 2: The dataset after the 'Removing Duplicate Data'.

⁴ Step 3: The dataset after the 'Removing Data with Missing CWE Identifiers'.

⁵ Step 4: The dataset after the 'Removing Functions with Empty Bodies or Only a Return Statement'.

⁶ Step 5: The dataset after the 'Removing Functions Based on Rules', i.e., the final derived dataset.

In the table, the "Vul" and "No-Vul" columns represent the number of vulnerable and non-vulnerable samples, respectively, while the "Total" column denotes their sum. The columns "CWE" and "Project" indicate the number of CWE types and the number of associated projects.

In addition, we invite five experienced programmers to manually inspect sampled data before and after each derivation step. The results show that the data derivation process is correct, the outcomes align with expectations, and the overall quality of the dataset improves after derivation.

3) *Evaluation Metrics*: To comprehensively evaluate the performance of our proposed method, we use widely recognized performance metrics, including Binary-Precision (BP), Binary-Recall (BR), Binary-F1 score (BF1), Macro-F1 score (MF1), and Matthews correlation coefficient (MCC). These metrics are derived from the confusion matrix, which consists of true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN). Specifically: BP reflects the proportion of “correctly predicted positive samples” out of all “predicted positive samples” and is calculated as: $BP = \frac{TP}{TP+FP}$. Higher precision indicates a greater likelihood that the predicted vulnerable samples actually contain vulnerabilities. BR measures the proportion of “correctly identified positive samples” out of all “actual positive samples” and is given by: $BR = \frac{TP}{TP+FN}$. Higher recall indicates that the model can detect more vulnerabilities. We also use the BF1 score, which balances precision and recall and is calculated as: $BF1 = 2 * \frac{BP*BR}{BP+BR}$. Since the above metrics focus only on the positive class and do not account for negative examples, we also use MF1 and MCC. MF1 computes the BF1 score for each class individually and then averages them, with the formula: $MF1 = \frac{F1_0+F1_1}{2}$. MF1 offers a comprehensive assessment of the model performance in all classes. MCC measures the correlation between the true and predicted labels, with the formula: $MCC = \frac{TP*TN-FP*FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$. A higher correlation between the true and predicted values indicates better prediction performance. MCC ranges from -1 to 1, where 1 indicates perfect prediction, 0 indicates random prediction, and -1 indicates a completely inverse prediction.

ACC is not used as an evaluation metric in this study because the data samples are highly imbalanced in real-world CVD and a model can achieve a high ACC simply by predicting all results as non-vulnerable.

4) *Baselines*: To establish a robust performance benchmark, we carefully select LMs that are widely used in the field of Natural Language Processing (NLP) as our baseline (some studies [5] refer to them as LLMs, but in this study, we use LMs to describe them). The details are as follows:

At first, we select two models that are widely adopted as standard baselines in multiple studies:

- microsoft-codebert-base: This model [46] is pre-trained on the CodeSearchNet dataset [58] using two objectives: Masked Language Modeling (MLM) and Replaced Token Detection (RTD).
- microsoft-codebert-base-mlm: This model is similar to microsoft-codebert-base but uses only the MLM objective for pre-training. Many researchers further train and optimize it for various tasks.

Since this study focuses on C/C++ real-world CVD, we specifically select two language models that are pre-trained on large-scale C/C++ code datasets:

- codebert-c: This model [52] is based on microsoft-codebert-base-mlm and is additionally trained on the C code using the MLM objective.
- codebert-cpp: This model is also based on microsoft-codebert-base-mlm and is additionally trained on the C++ code using the MLM objective.

Besides CodeBert-based models, we also select models that achieve state-of-the-art results on CodeXGLUE benchmark:

- codet5-base: This model [50] is specifically designed for code understanding and generation tasks. It is pre-trained on the CodeSearchNet dataset and C/Sharp data collected from BigQuery1.
- codet5-small: This is a simplified version of codet5-base with fewer parameters, making it more lightweight and suitable for resource-constrained environments.

In addition to code-related pre-trained models, we also use the popular text embedding model FlagEmbedding, which outperforms other similar models in most tasks. The specific model we use is the following.

- bge-base-en-v1.5: This model [49] is a special version of FlagEmbedding, which has a more reasonable similarity distribution compared to the original version.

Beyond the language models mentioned above, we also select two multi-stage pipeline baselines that are similar to our framework. These baselines also utilize intermediate structural representations of code and involve extensive preprocessing. During training, we adjust their class weights to improve performance. The specific models we use are the following.

- TrVD: This model [17] represents the current state-of-the-art (SOTA) performance in vulnerability existence detection and precise vulnerability type identification. It leverages AST, tree-structured neural networks, and Transformer-style encoders to detect vulnerabilities.
- Mocktail: This model [37] previously demonstrated SOTA performance in method naming, program classification, and clone detection. We adapt it for vulnerability detection and conduct experiments accordingly. The model analyzes various code intermediate structures (AST, CFG, CDG, and DDG) and extracts path-based features to support subsequent software engineering tasks.

In order to maintain practical usability and transparency in the evaluation, all language models used in this study are sourced from the Hugging Face model hub, and all multi-stage pipelines are open source.

5) *Parameter Settings*: When fine-tuning the LMs, we set *epoch*=10 (since all models achieve good performance within 10 epochs), *block_size*=512, *learning_rate*=1e-5, and *ma_grad_norm*=1.0. Considering the large parameter size of codet5-base, we set its batch size to 4, codeT5-small to 8, and all other models to 16.

When training other baselines, we use their default parameters and only add the corresponding class weights for training.

When training our models, We set *epoch*=30 (*early_stop*=10), *batch_size*=512, and *learning_rate*=1e-2 (with *step_size*=1 and *gamma*=0.8).

C. Experiments for Answering RQ1

Directly comparing the quality of the dataset before and after derivation is challenging due to the large size of the dataset. Therefore, we adopt an indirect evaluation method: we fine-tuned the microsoft-codebert-base model on the original large dataset (excluding the original testing set and the derived

testing set) and then evaluated it separately on the original testing set and the derived testing set. The evaluation results are shown in Table III.

TABLE III
COMPARISON OF THE TEST EFFECT OF THE SAME MODEL ON TWO TESTING SETS.

Dataset	Weight	BP	BR	BF1	MF1	MCC
OriTSet ¹	1	0.2541	0.1778	0.2092	0.5848	0.1739
DerTSet ²	1	0.3339	0.2349	0.2758	0.5602	0.1289
OriTSet	20	0.2805	0.241	0.2592	0.6092	0.2194
DerTSet	20	0.3653	0.2903	0.3235	0.5844	0.1729

¹ OriTSet: The original dataset that has not undergone data derivation.

² DerTSet: The Derived dataset that has undergone data derivation.

It is important to note that the training set is not augmented and remains imbalanced. We also compare test results with and without class weights (weight set to 20 to match the original class ratio). The ‘Weight’ column in the table III represents the weights applied during the calculation of cross-entropy loss.

The evaluation results show that the new testing set significantly outperforms the original testing set in BP, BR, and BF1, indicating that the new dataset contains less noise. However, its MF1 and MCC scores are slightly lower. We attribute this to two main reasons: on the one hand, the severe class imbalance in the original training set causes the model to favor predicting “non-vulnerable” during fine-tuning, which aligns with the higher proportion of such samples in the original test set; on the other hand, the distinction between non-vulnerable and vulnerable samples is more pronounced in the original testing set, making predictions easier for the model. Nevertheless, since BP, BR, and BF1 are generally better indicators of real-world detection performance, we believe the new dataset is overall superior in terms of data distribution and noise level.

In subsequent research, we conduct experiments based on the derived dataset, DiverseVul-Derived.

D. Experiments for Answering RQ2

The simplification of code intermediate structures and data augmentation are key components of MARCOVul. The intermediate structure simplification method is specific to AST, CFG, and PDG. It aims to reduce graph size, lower resource consumption and computational complexity, and better leverage LMs to extract rich structural features. The data augmentation technique is applied to both the intermediate structures and the raw code, using different methods to balance the number of positive and negative samples in the training set.

To systematically evaluate the impact of intermediate structure simplification and data augmentation on real-world CVD, we design and conduct four experiments. These experiments train and test models on the following datasets: the original dataset, the dataset with intermediate structure simplification, the dataset with data augmentation, and the dataset with both intermediate structure simplification and data augmentation.

Since intermediate structure simplification only affects the intermediate representations of the code and not the original code, the related experiments are performed based solely on

the code structure. To ensure fairness, all experiments use the widely used GCN model, with parameter settings following standard parameters, and the code embeddings are generated using the microsoft-codebert-base model. No weights are introduced in the calculation of the loss function. The experimental results are presented in Table IV.

TABLE IV
THE INFLUENCE OF INTERMEDIATE STRUCTURE SIMPLIFICATION AND DATA AUGMENTATION ON DETECTION RESULTS.

Dataset	BP	BR	BF1	MF1	Mcc
Orig¹	0.32927	0.03321	0.06034	0.46959	0.04390
Simp²	0.33588	0.10824	0.16372	0.51538	0.08474
Aug³	0.32341	0.20049	0.24753	0.54776	0.10982
Simp-Aug⁴	0.28838	0.32349	0.30493	0.55436	0.10985

¹ Dataset with no operations performed.

² Dataset with intermediate structure simplification.

³ Dataset with data augmentation.

⁴ Dataset with both intermediate structure simplification and data augmentation.

The experimental results show that, except for BP, the fusion of intermediate structure simplification and data augmentation(*Simp-Aug*) outperforms the approach that only applies data augmentation(*Aug*) in terms of real-world CVD. The *Aug* is superior to the approach that only simplifies the intermediate structure(*Simp*), while *Simp* performs better than the original data(*Orig*). These results indicate that both *Simp* and *Aug* contribute to improving the real-world CVD performance. In particular, *Aug* has a more significant impact on performance improvement. For example, in the MF1 and MCC metrics, the performance of *Aug* is already close to *Simp-Aug*, while the performance improvement brought by *Simp* is relatively small.

To further verify the reliability of these findings, we adjust the experimental parameters and conduct extensive experiments. The results show that *Simp-Aug* consistently outperforms *Orig* and *Simp* in terms of performance, but in some cases, its performance is slightly lower than *Aug*. This is due to the lower stability of *Simp*, which sometimes even performs worse than *Orig*. This phenomenon is intuitive because, although intermediate structure simplification reduces the number of nodes and enables more effective utilization of the language model, the merging of nodes and the removal of edges inevitably lead to the loss of some internal structural information, which may cause the performance of *Simp* to be merely comparable to, or even worse than, that of *Orig*. The effectiveness of *Simp* essentially depends on the relative dominance between the negative impact of structural information loss and the positive effect of language model utilization under different scenarios and parameter configurations. Experimental results show that *Simp* improves the detection performance to some extent by accelerating the training process and making better use of the language model. However, its overall performance stability is affected, resulting in significant fluctuations.

To better evaluate the effectiveness of our simplification method, we conduct additional experiments on a smaller codebase, DiverseVul-Derived-3K, which consists of samples from DiverseVul-Derived whose corresponding.pkl files are

smaller than 3KB. This dataset contains 5,014 non-vulnerable functions and 701 vulnerable functions in the training set, 664 / 72 in the validation set, and 620 / 65 in the testing set. We apply our simplification method to this dataset to create a comparison dataset called DiverseVul-Derived-3K-Simp. By comparing the experimental results on these two datasets, we evaluate the effectiveness and information retention capability of our simplification method on smaller codebases.

In the experiments, we also uniformly use the GCN model to extract graph features from the intermediate structures and the microsoft-codebert-base model to embed the code. However, this small dataset is highly imbalanced (5,014:701 in the training set). If we do not apply weight to the loss function, most evaluation metrics drop to zero. Therefore, we introduce a class weight of 7, calculated by flooring the sample ratio in the training set. This highlights the necessity of using our data augmentation method to mitigate sample imbalance during training. The experimental results are shown in Table V.

TABLE V
THE INFLUENCE OF INTERMEDIATE STRUCTURE SIMPLIFICATION ON
DETECTION RESULTS OF SMALLER CODEBASE.

Dataset	BP	BR	BF1	MF1	Mcc
DivDer-3K ¹	0.1935	0.1846	0.189	0.5531	0.1062
DivDer-3K-Simp ²	0.3231	0.178	0.2295	0.5554	0.1293

¹ DivDer-3K represents the dataset DiverseVul-Derived-3K, which is composed of all the data selected from the dataset DiverseVul-Derived where the size of pk1 files is less than 3KB.

² DivDer-3K-Simp represents the dataset obtained by applying our data simplification method to DiverseVul-Derived-3K.

The experimental results show that the simplified dataset performs better in most evaluation metrics. This indicates that the positive effect of leveraging the language model outweighs the negative effect of losing some structural information in this case. We believe that the strong representation capability of the language model enables it to capture part of the structural information lost during simplification, which explains why the positive effect of the language model surpasses the negative impact of structural information loss in most cases. Therefore, we believe that our data simplification method has good information retention capability.

Considering the overall results, *Simp-Aug* generally achieves the best detection performance. Therefore, we conclude that both intermediate structure simplification and data augmentation are effective methods. In subsequent experiments, our model will focus on using data processed with *Simp-Aug* to achieve the best real-world CVD performance.

E. Experiments for Answering RQ3

To validate the effectiveness of MARCOVul in real-world vulnerability detection, we select seven large-scale pre-trained language models and two multi-stage pipeline models as baselines. Additionally, to explore the impact of code normalization and weight setting, we conduct experiments on each LM baseline with four different configurations (with / without normalization and with / without class weights). For the two multi-stage pipeline models, we also conduct experiments

with class weight adjustments. It is important to note that MARCOVul does not rely on these additional operations. Detailed experimental results are presented in Table VI.

Analysis of the baseline results shows that under the same weight configuration, different models exhibit relatively small performance differences. However, the introduction of weights has a significant impact on the detection performance. Without weights, models generally perform poorly, while enabling weights substantially improves key metrics such as BR, BF1, MF1, and MCC. Although the use of weights causes slight fluctuations in BP, the magnitude of this change is acceptable. This is because, without weights, models tend to predict functions as non-vulnerable (resulting in high BP and low BR), whereas weights shift the focus toward vulnerable functions, improving BR at the cost of some BP. These findings directly highlight the necessity of data augmentation in our framework.

In addition, code normalization also affects detection performance, potentially causing a slight decrease. This is because normalization replaces variable and function names, which may result in the loss of some critical semantic information that LMs rely on for detection. This finding highlights the importance of code semantics in the detection process.

By comparing models before and after large-scale fine-tuning on specific programming languages, we find that such fine-tuning does not significantly improve vulnerability detection capabilities, while weight configuration has a greater impact. Similarly, the performance difference between codeT5-base and codeT5-small is minimal, indicating that the model scale plays a less critical role than the weight settings. These results suggest that in practical applications, general-purpose models remain effective when fine-tuned models for specific languages are unavailable, and smaller models are a feasible option when device performance is limited.

By comparing the vulnerability detection results of MARCOVul with all baselines, we observe that MARCOVul outperforms these baseline models on most metrics. This demonstrates that MARCOVul can effectively improve the performance of real-world CVD. MARCOVul integrates the structural features of the code's AST, CFG, and PDG with the semantic features of the text and combines these features to enhance real-world CVD performance. Before feature extraction, the AST, CFG, and PDG have been processed by structural simplification and data augmentation, and the text has been processed by data augmentation.

To validate the effectiveness of each feature used in MARCOVul, we conduct ablation experiments. The experimental results are shown in Table VII.

According to the experimental results, we find that integrating four types of code representations achieves the best detection performance. However, compared to using a single text representation (Text), the performance improvement brought by integrating the four representations (All Feature) is not significant. It is worth noting that under the same parameter settings, the detection performance of the single text representation already outperforms the method that only integrates three intermediate structural representations (AST+CFG+PDG) in terms of BP, BF1, MF1, and Mcc.

TABLE VI
COMPARISON OF VULNERABILITY DETECTION RESULTS BETWEEN MARCOVUL AND BASELINE¹.

Model	Normalization ²	Weight ³	BP	BR	BF1	MF1	Mcc
microsoft-codebert-base	NO	1	0.339	0.2202	0.267	0.5578	0.128
microsoft-codebert-base	YES	1	0.3589	0.1845	0.2437	0.5514	0.129
microsoft-codebert-base-mlm	NO	1	0.3434	0.2522	0.2908	0.5676	0.1421
microsoft-codebert-base-mlm	YES	1	0.3613	0.2066	0.2629	0.5596	0.1392
codebert-c	NO	1	0.3501	0.2312	0.2785	0.5642	0.1401
codebert-c	YES	1	0.327	0.2964	0.311	0.5705	0.1418
codebert-cpp	NO	1	0.3522	0.2066	0.2605	0.5574	0.1329
codebert-cpp	YES	1	0.346	0.2239	0.2718	0.5609	0.1345
codet5-small	NO	1	0.5066	0.1427	0.2226	0.5517	0.1837
codet5-small	YES	1	0.4917	0.0726	0.1265	0.5042	0.1246
codet5-base	NO	1	0.397	0.1968	0.2632	0.5639	0.1582
codet5-base	YES	1	0.4113	0.1968	0.2662	0.5666	0.1669
bge-base-en-v1.5	NO	1	0.2898	0.2128	0.2454	0.5399	0.086
bge-base-en-v1.5	YES	1	0.3222	0.1784	0.2296	0.5408	0.1019
TrVD	Special ⁴	1	0.6875	0.0562	0.1039	0.4958	0.1558
Mocktail	Special	1	0.2431	0.1608	0.1935	0.5011	0.0124
microsoft-codebert-base	NO	3	0.3806	0.4256	0.4019	0.6167	0.2347
microsoft-codebert-base	YES	3	0.3673	0.4477	0.4035	0.6126	0.2293
microsoft-codebert-base-mlm	NO	3	0.3755	0.4526	0.4105	0.6178	0.2393
microsoft-codebert-base-mlm	YES	3	0.3435	0.4723	0.3977	0.6004	0.2119
codebert-c	NO	3	0.3489	0.4785	0.4035	0.6044	0.2199
codebert-c	YES	3	0.3537	0.4834	0.4085	0.608	0.2269
codebert-cpp	NO	3	0.374	0.4108	0.3916	0.6111	0.2231
codebert-cpp	YES	3	0.3638	0.4207	0.3902	0.6072	0.2166
codet5-small	NO	3	0.349	0.4846	0.4058	0.605	0.222
codet5-small	YES	3	0.3362	0.5289	0.4111	0.5983	0.2209
codet5-base	NO	3	0.3764	0.4268	0.4	0.6147	0.231
codet5-base	YES	3	0.3444	0.4957	0.4065	0.6027	0.2203
bge-base-en-v1.5	NO	3	0.3173	0.6248	0.4209	0.5815	0.2251
bge-base-en-v1.5	YES	3	0.3175	0.6335	0.423	0.5814	0.228
TrVD	Special	3	0.3774	0.3972	0.3871	0.6113	0.223
Mocktail	Special	3	0.3124	0.6516	0.4223	0.5514	0.1866
MARCOVul	Partial ⁵	1	0.3378	0.6248	0.4385	0.6014	0.2535

¹ All experiments are conducted based on DiverseVul-Derived dataset.

² The Normalization column indicates whether code normalization has been applied.

³ The Weight column specifies the weight value used in the loss function computation(1 or 3), which is determined based on the ratio of non-vulnerable to vulnerable samples.

⁴ The "Special" indicates that these methods use their own normalization.

⁵ The "Partial" indicates that normalization is applied only to a subset of features, specifically the intermediate structural features, while the code text features are not normalized.

TABLE VII
THE ABLATION EXPERIMENT OF FEATURES USED IN MARCOVUL.

Feature	BP	BR	BF1	MF1	Mcc
AST	0.3371	0.3284	0.3327	0.5804	0.1609
CFG	0.2890	0.6900	0.4074	0.5431	0.1992
PDG	0.2911	0.5006	0.3682	0.5599	0.1536
AST+CFG+PDG	0.2895	0.7737	0.4213	0.5325	0.2255
Text	0.3288	0.6285	0.4318	0.5929	0.2426
All Feature ¹	0.3378	0.6248	0.4385	0.6014	0.2535

¹ The All Feature includes AST, CFG, PDG, and TEXT.

We attribute this phenomenon primarily to the following two reasons:

- **Semantic loss caused by intermediate structure normalization:** Although the normalization of code aims to minimize the impact of individual coding styles on the extraction of structural features, it inevitably leads to the loss of semantic information. Some vulnerabilities can only be identified through semantic features, which are

removed or obscured during the normalization process. As a result, relying solely on normalized intermediate structures makes it difficult to detect such vulnerabilities. This limitation underscores the importance of incorporating raw text representation into our framework.

- **Strong feature extraction capability of language models:** All code intermediate structures are generated from the code text and essentially represent an internal structural form of the code. Powerful language models (e.g., the models adopted in this work) can directly extract internal structural features similar to intermediate structures without relying on them. In our preliminary experiments, we evaluated a Word2Vec+RNN approach to extract text features, which performed significantly worse than methods using intermediate structures. This indicates that the superior performance of our single text representation, compared to the fusion of three intermediate structures, is primarily due to the strong feature extraction capability of the language model, which captures implicit structural patterns similar to intermediate structure representations.

Nevertheless, since current language models are not yet capable of fully replacing intermediate structures, the latter can still serve as complementary features to improve vulnerability detection performance. However, the improvement brought by this supplementation is limited. The approach that combines intermediate structures with code text features yields only marginal gains compared to using code text features alone.

To further explore the sensitivity of the model to parameter variations, we conduct extensive hyperparameter tuning experiments. The results show that the detection performance based on code intermediate structures is highly sensitive to parameters such as the number of model layers, number of attention heads, and learning rate. When the parameter configuration is suboptimal (e.g., a mismatch between the learning rate and the number of model layers or attention heads), using only the three intermediate structures performs significantly worse than using the text representation alone. In such cases, even combining all four code representations fails to outperform the single text representation.

Therefore, we conclude that the fusion of four different code representations can lead to better detection results, but the training process is more difficult and requires longer training times. If the goal is to achieve relatively good detection performance, directly fine-tuning the language model on the augmented raw code would be a more suitable approach.

F. Experiments for Answering RQ4

The three code intermediate structures (AST, CFG, PDG) involved in real-world CVD represent different structural features of the code. MARCOVul hypothesizes that different code representations require different machine learning models for effective feature extraction. To verify the effectiveness of capturing different code representations using different models, we conduct the following experiments.

We select ten representative models to extract features from the three intermediate structures. The selected models include: Tree-LSTM(TLSTM), GCN, GIN, GAT, GNN-FILM(FILM), Gaussian Mixture Model (GMM), Transformer(Trans), Topology Adaptive Graph Convolution Network(TAG), Residual Gated Graph Convolution (RGGConv), and GraphSAGE(SAGE). These models are selected because they have been frequently used in previous studies to extract structure features of code and are widely applied in tasks such as graph feature extraction, vulnerability detection, and code repair. In all experiments, each model is configured with a standard three-layer architecture. This design follows findings from existing studies and our experiments, which show that a three-layer setting typically provides strong performance, while deeper configurations offer diminishing returns or even degrade effectiveness. For models that support multi-head or multi-kernel configurations, we explore different head (kernel) configurations and select the best-performing results based on available hardware resources.

Due to the large number of evaluation metrics, we use the following formula to calculate a total score(TS):

$$TS = \alpha * BP + \beta * BR + \gamma * BF1 + \delta * MF1 + \epsilon * MCC \quad (9)$$

where $(\alpha, \beta, \gamma, \delta, \epsilon)$ are variable coefficients. In this experiment, they are assigned values based on the importance of the metrics as follows: (0.1, 0.1, 0.2, 0.3, 0.3).

The experimental results for AST are shown in Figure 10:

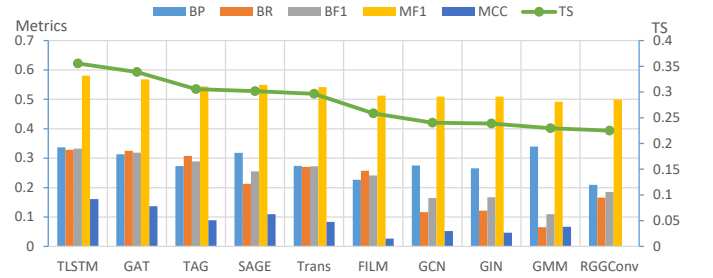


Fig. 10. The influence of different models on AST feature capture.

The analysis result shows that, for AST, the TreeLSTM model performs the best. To verify the reliability of the result, we adjust the model parameters and conduct extensive experiments. The experimental results show that, although GAT performs slightly better than TreeLSTM under certain parameter configurations, GAT has lower stability, with its performance drastically declining under other configurations. In contrast, TreeLSTM demonstrates more stable detection results. Therefore, we believe that TreeLSTM is more suitable for handling AST data and can more effectively capture relevant features. This conclusion aligns with our intuition, since tree structures have a clear hierarchical feature, whereas graph structures do not. Applying graph methods to tree structures may lose some hierarchical information, thereby affecting the final detection results.

The experimental results for CFG are shown in Figure 11:

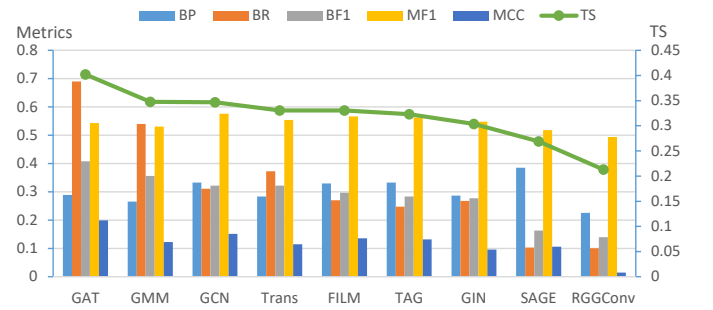


Fig. 11. The influence of different models on CFG feature capture.

The analysis results show that, for CFG, the GAT model performs the best. We also adjust the parameters and conduct extensive experiments. The experimental results show that, for CFG, GAT significantly outperforms other models in most parameter configurations. We analyze that CFG consists of basic blocks in the code as nodes and control flow transfers as edges. GAT uses the attention mechanism to compute the relevance between nodes, which is crucial for understanding the relationships and state transitions between nodes in the CFG. This allows for the effective identification of abnormal control flows that lead to vulnerabilities.

The experimental results for PDG are shown in Figure 12:

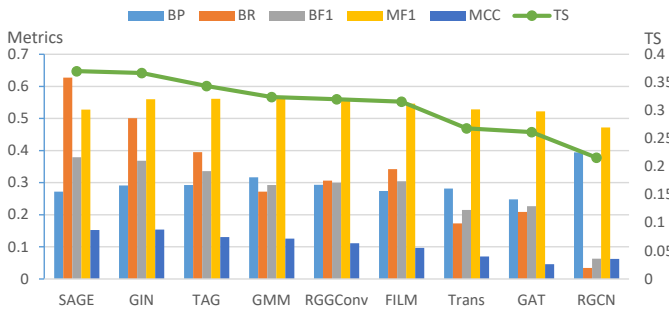


Fig. 12. The influence of different models on PDG feature capture.

The analysis results show that, for PDG, the GIN and GraphSAGE models perform the best. To verify the stability of the experimental results, we adjust the parameters and conduct extensive experiments. The results indicate that GIN outperforms other models in most cases. Although in certain parameter configurations, GAT and GraphSAGE perform similarly to GIN, their detection effects are unstable, and in some configurations, their performance is even poor. For GAT, aside from its instability, the larger scale of PDG also leads to higher memory usage during training. We analyze that the reason GIN outperforms other models is that PDG is a program dependency graph, and its dependencies can be seen as a representation of program structure, which GIN is specifically designed to differentiate in graph structures. Although GAT has advantages in capturing graph structures, its memory usage is extremely high, especially when using RGAT, and its performance is unstable. In contrast, GIN has lower memory usage and more stable performance.

Through experimental validation, we conclude that for the diverse intermediate structural features of the code, diverse models are required to capture them. A single model is insufficient to comprehensively and effectively extract all the features of the intermediate structures.

V. DISCUSSION

A. Impact of code intermediate structure simplification

Our code intermediate structure simplification approach reduces the graph structure by removing edges between nodes with the same $\langle SUB \rangle x \langle /SUB \rangle$ attribute and merging these nodes into supernodes, while preserving the original semantic information. On the one hand, this simplification inevitably causes the loss of some internal structural information, but it does not compromise key semantic information, as we retain the complete raw text. On the other hand, by keeping the raw text, we better leverage the language model's ability to capture contextual information. Moreover, the powerful language model can partially learn the structural information lost during the simplification process.

The extensive experimental results in Section IV-D (Experiments for Answering RQ2) indicate that graph simplification produces unstable effects. We believe that this instability stems from two factors: on one hand, the simplification process inevitably loses some internal structural information due to node merging and edge removal; on the other hand, simplification enables the language model to more effectively capture

contextual information and may even allow the model to infer part of the lost structure indirectly. The trade-off between the negative impact of structural information loss and the positive impact of language model utilization varies across different scenarios and parameter settings, directly influencing the final detection performance. Nevertheless, since graph simplification significantly reduces graph size and facilitates better utilization of the language model, and our experiments show that it improves vulnerability detection performance to some extent, we consider graph simplification an effective approach and adopt it in our framework.

B. Application of MARCOVul in practical engineering

In practical engineering scenarios, resource consumption and timeliness are critical factors in determining usability. Our study shows that combining multiple intermediate structures with raw code text achieves the best vulnerability detection performance, but using only raw code text also provides relatively strong results. This is because intermediate structures represent the inherent structure of the code, and if the language model is sufficiently capable, it can detect vulnerabilities directly from the text without relying on these structures. Although relying solely on text representation cannot fully capture all code features yet, it already achieves near-optimal results. Regarding runtime efficiency and resource consumption, our tests show that using all four feature types typically requires 2 to 10 times more time than using only the single text feature and consumes more computational resources (approximately 1.5 GB of additional GPU memory). However, even with all four features, the average processing time per data sample does not exceed 0.25 seconds, which we consider acceptable. This is critical for real-world CVD because it allows users to choose different solutions based on available resources and time constraints. When resources and time are sufficient, combining intermediate structures with raw code text achieves the best results; when resources and time are limited, using only the raw code text still produces good results. Our framework provides corresponding configuration files, allowing users to choose which features to use for vulnerability detection based on their needs, such as using all four features or only the text feature.

C. Generalizability of MARCOVul in other programming languages

We believe that our framework (MARCOVul) can be easily extended to other programming languages. This is because MARCOVul takes raw source code as its initial input, and the features it relies on, such as AST, CFG, PDG, and Text, are inherently cross-language and not limited to C/C++. The Joern that we use already supports parsing 19 programming languages, including the major ones such as C, C++, C#, Go, Java, Python, and PHP. Therefore, applying this approach to other languages is straightforward and primarily requires collecting the corresponding training data for model training.

VI. THREATS TO VALIDITY

Although we carefully design and conduct the experiments, there are still several threats to the validity of this study. These threats to validity are summarized below.

A. External validity

The external validity of this study is primarily constrained by the dataset used in the experiments. Specifically, we rely solely on the DiverseVul dataset, excluding other datasets such as BigVul, Devign, and Reveal. This decision is based on the following considerations: DiverseVul is currently one of the most comprehensive datasets with a relatively low error rate. Additionally, it significantly overlaps with these alternative datasets, covering the majority of their data. Therefore, we consider using this dataset alone to be sufficient.

B. Internal validity

The internal validity of this study is primarily influenced by the quality of the dataset. Currently, the quality of publicly available real-world code vulnerability datasets is relatively poor. We address these issues as much as possible through the data derivation method proposed in MARCOVul.

VII. CONCLUSION

The framework (MARCOVul) proposed in this study provides a comprehensive and efficient solution for detecting vulnerabilities in real-world code. MARCOVul first introduces a data derivation method to enhance the quality of the experimental dataset. It then generates the intermediate structures of the code (AST, CFG, PDG) and proposes an intermediate structure simplification method to reduce computational complexity and resource consumption while fully leverage the capabilities of language models. At the same time, to address the class imbalance issue in real-world CVD, a code-specific data augmentation method is proposed, which performs data augmentation on all features of the code. In the process of extracting code vulnerability features, different models are used for different code structures to extract the corresponding features. Finally, an attention-based multi-feature fusion classifier is utilized to classify whether the code contains vulnerabilities. MARCOVul optimizes the entire process from data preprocessing to final vulnerability detection, improving the effectiveness of real-world CVD. On the large-scale real-world CVD dataset DiverseVul-Derived, MARCOVul achieves the best overall performance, improving the best BF1 and MCC scores of all unweighted baselines by 12.75% and 6.98%, respectively, and outperforming the best weighted baselines by 1.55% and 1.42%, respectively. However, MARCOVul does not achieve significant improvements in all research questions (RQs). The performance gain from multi-feature fusion over the single text representation is relatively limited. Nonetheless, the single text representation within our framework still outperforms the current best baselines.

The derived dataset constructed in this work contains relevant CWE labels and ensures a reasonable data distribution. This dataset can be used in future research to develop multinomial classification models, aiming to classify detected code vulnerabilities into specific CWE types.

VIII. ACKNOWLEDGMENT

This work was supported by National Science and Technology Major Project of China (Grant No. 2022ZD0120303).

REFERENCES

- [1] "CWE-2023 CWE Top 25 Most Dangerous Software Weaknesses." <https://cwe.mitre.org/data/definitions/1425.html>. [Accessed 13-01-2025].
- [2] "CWE-2024 CWE Top 25 Most Dangerous Software Weaknesses." https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html. [Accessed 13-01-2025].
- [3] Z. Li, Z. Liu, W. K. Wong, P. Ma, and S. Wang, "Evaluating c/c++ vulnerability detectability of query-based static application security testing tools," *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [4] B. Casey, J. Santos, and G. Perry, "A survey of source code representations for machine learning-based cybersecurity tasks," *arXiv preprint arXiv:2403.10646*, 2024.
- [5] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, pp. 654–668, 2023.
- [6] Y. Guo, S. Bettaieb, and F. Casino, "A comprehensive analysis on software vulnerability detection datasets: trends, challenges, and road ahead," *International Journal of Information Security*, vol. 23, no. 5, pp. 3311–3327, 2024.
- [7] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "Deepwukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 3, pp. 1–33, 2021.
- [8] K. Napier, T. Bhowmik, and Z. Chen, "Explaining poor performance of text-based machine learning models for vulnerability detection," *Empirical Software Engineering*, vol. 29, no. 5, p. 113, 2024.
- [9] "NIST Software Assurance Reference Dataset — samate.nist.gov." <https://samate.nist.gov/SARD/>. [Accessed 19-09-2024].
- [10] X. Duan, J. Wu, S. Ji, Z. Rui, T. Luo, M. Yang, and Y. Wu, "Vulsniper: Focus your attention to shoot fine-grained vulnerabilities," in *IJCAI*, pp. 4665–4671, 2019.
- [11] D. Cao, J. Huang, X. Zhang, and X. Liu, "Ftclnet: Convolutional lstm with fourier transform for vulnerability detection," in *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pp. 539–546, IEEE, 2020.
- [12] H. Feng, X. Fu, H. Sun, H. Wang, and Y. Zhang, "Efficient vulnerability detection based on abstract syntax tree and deep learning," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 722–727, IEEE, 2020.
- [13] C. Zhang, B. Liu, Y. Xin, and L. Yao, "Cpvd: Cross project vulnerability detection based on graph attention network and domain adaptation," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4152–4168, 2023.
- [14] B. Wu, F. Zou, P. Yi, Y. Wu, and L. Zhang, "Slicedlocator: Code vulnerability locator based on sliced dependence graph," *Computers & Security*, vol. 134, p. 103469, 2023.
- [15] J. Gear, Y. Xu, E. Foo, P. Gauravaram, Z. Jadidi, and L. Simpson, "Software vulnerability detection using informed code graph pruning," *IEEE Access*, vol. 11, pp. 135626–135644, 2023.
- [16] Y. Fan, C. Wan, C. Fu, L. Han, and H. Xu, "Vdotr: Vulnerability detection based on tensor representation of comprehensive code graphs," *Computers & Security*, vol. 130, p. 103247, 2023.
- [17] Z. Tian, B. Tian, J. Lv, Y. Chen, and L. Chen, "Enhancing vulnerability detection via ast decomposition and neural sub-tree encoding," *Expert Systems with Applications*, vol. 238, p. 121865, 2024.
- [18] W. Xiao, Z. Hou, T. Wang, C. Zhou, and C. Pan, "Msgvul: Multi-semantic integration vulnerability detection based on relational graph convolutional neural networks," *Information and Software Technology*, p. 107442, 2024.
- [19] L. Kong, S. Luo, L. Pan, Z. Wu, and X. Li, "A multi-type vulnerability detection framework with parallel perspective fusion and hierarchical feature enhancement," *Computers & Security*, vol. 140, p. 103787, 2024.
- [20] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "Vuldelocator: a deep learning-based fine-grained vulnerability detector," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2821–2837, 2021.

- [21] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.
- [22] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?," *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2021.
- [23] G. Nikitopoulos, K. Dritsa, P. Louridas, and D. Mitropoulos, "Crossvul: a cross-language vulnerability dataset with commit data," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1565–1569, 2021.
- [24] G. Bhandari, A. Naseer, and L. Moonen, "Cvefixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, pp. 30–39, 2021.
- [25] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A c/c++ code vulnerability dataset with code changes and cve summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories*, pp. 508–512, 2020.
- [26] C. Ni, L. Shen, X. Yang, Y. Zhu, and S. Wang, "Megavul: Ac/c++ vulnerability dataset with comprehensive code representations," in *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, pp. 738–742, IEEE, 2024.
- [27] W. Tang, M. Tang, M. Ban, Z. Zhao, and M. Feng, "Csgvd: A deep learning approach combining sequence and graph embedding for source code vulnerability detection," *Journal of Systems and Software*, vol. 199, p. 111623, 2023.
- [28] Q. Wang, Z. Li, H. Liang, X. Pan, H. Li, T. Li, X. Li, C. Li, and S. Guo, "Graph confident learning for software vulnerability detection," *Engineering Applications of Artificial Intelligence*, vol. 133, p. 108296, 2024.
- [29] C. X. Do, N. T. Luu, and P. T. L. Nguyen, "Optimizing software vulnerability detection using roberta and machine learning," *Automated Software Engineering*, vol. 31, no. 2, p. 40, 2024.
- [30] R. Wang, S. Xu, Y. Tian, X. Ji, X. Sun, and S. Jiang, "Scl-cvd: Supervised contrastive learning for code vulnerability detection via graphcodebert," *Computers & Security*, vol. 145, p. 103994, 2024.
- [31] A. Narayanan, L. Yang, L. Chen, and L. Jinliang, "Adaptive and scalable android malware detection through online learning," in *2016 International Joint Conference on Neural Networks (IJCNN)*, pp. 2484–2491, IEEE, 2016.
- [32] A. Narayanan, M. Chandramohan, L. Chen, and Y. Liu, "Context-aware, adaptive, and scalable android malware detection through online learning," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 1, no. 3, pp. 157–175, 2017.
- [33] A. Narayanan, M. Chandramohan, L. Chen, and Y. Liu, "A multi-view context-aware approach to android malware detection and malicious code localization," *Empirical Software Engineering*, vol. 23, pp. 1222–1274, 2018.
- [34] J. Gear, Y. Xu, E. Foo, P. Gauravaram, Z. Jadidi, and L. Simpson, "Scevd: Semantic-enhanced code embedding for vulnerability discovery," in *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pp. 1522–1527, IEEE, 2022.
- [35] Y. Wu, J. Lu, Y. Zhang, and S. Jin, "Vulnerability detection in c/c++ source code with graph representation learning," in *2021 IEEE 11th Annual Computing and Communication Workshop and Conference (CCWC)*, pp. 1519–1524, IEEE, 2021.
- [36] W. Wang, T. N. Nguyen, S. Wang, Y. Li, J. Zhang, and A. Yadavally, "Deepvd: Toward class-separation features for neural network vulnerability detection," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 2249–2261, IEEE, 2023.
- [37] K. C. Swarna, N. S. Mathews, D. Vagavolu, and S. Chimalakonda, "On the impact of multiple source code representations on software engineering tasks—an empirical study," *Journal of Systems and Software*, vol. 210, p. 111941, 2024.
- [38] S. Salimi and M. Kharrazi, "Vulslicer: Vulnerability detection through code slicing," *Journal of Systems and Software*, vol. 193, p. 111450, 2022.
- [39] M. Pradel and K. Sen, "Deepbugs: A learning approach to name-based bug detection," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–25, 2018.
- [40] G. Lin, J. Zhang, W. Luo, L. Pan, Y. Xiang, O. De Vel, and P. Montague, "Cross-project transfer representation learning for vulnerable function discovery," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 7, pp. 3289–3297, 2018.
- [41] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "Vuldeepecker: A deep learning-based system for vulnerability detection," *arXiv preprint arXiv:1801.01681*, 2018.
- [42] Q. Feng, C. Feng, and W. Hong, "Graph neural network-based vulnerability predication," in *2020 IEEE international conference on software maintenance and evolution (ICSME)*, pp. 800–801, IEEE, 2020.
- [43] H. Sun, L. Cui, L. Li, Z. Ding, S. Li, Z. Hao, and H. Zhu, "Vdtriple: Vulnerability detection with graph semantics using triplet model," *Computers & Security*, vol. 139, p. 103732, 2024.
- [44] C. Thapa, S. I. Jang, M. E. Ahmed, S. Camtepe, J. Pieprzyk, and S. Nepal, "Transformer-based language models for software vulnerability detection," in *Proceedings of the 38th Annual Computer Security Applications Conference*, pp. 481–496, 2022.
- [45] X. Zhou, D. Han, and D. Lo, "Assessing generalizability of codebert," in *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 425–436, IEEE, 2021.
- [46] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.
- [47] N. Pinnaparaju, R. Adithyan, D. Phung, J. Tow, J. Baicoianu, and N. Cooper, "Stable code 3b."
- [48] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, et al., "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [49] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C3-pack: Packaged resources to advance general chinese embedding," 2023.
- [50] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, 2021.
- [51] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, "Codet5+: Open code large language models for code understanding and generation," *arXiv preprint arXiv:2305.07922*, 2023.
- [52] S. Zhou, U. Alon, S. Agarwal, and G. Neubig, "Codebertscore: Evaluating code generation with pretrained models of code," *arXiv preprint arXiv:2302.05527*, 2023.
- [53] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "Sysevr: A framework for using deep learning to detect software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2244–2258, 2021.
- [54] K. S. Tai, R. Socher, and C. D. Manning, "Improved semantic representations from tree-structured long short-term memory networks," *arXiv preprint arXiv:1503.00075*, 2015.
- [55] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
- [56] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," *arXiv preprint arXiv:1810.00826*, 2018.
- [57] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "Lightgcn: Simplifying and powering graph convolution network for recommendation," *ACM*, 2020.
- [58] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.



Jixian Zhang received the B.E. degree in Software Engineering from Tongji University, China, in 2023, and is currently working toward the Eng.D. degree with the School of Computer Science and Technology, Tongji University, Shanghai, China. His current research interests include code vulnerability detection, software testing and AIOps.



Qingfeng Du received the Ph.D. degree from Tongji University, China, in 2013. He is currently a Professor with the School of Software Engineering, Tongji University, China. His main research interests include AIOps, software testing and big data.



Sheng Li received the B.E. degree in Software Engineering from Tongji University, China, in 2023, and is currently working toward the Ph.D. degree with the School of Computer Science and Technology, Tongji University, Shanghai, China. His current research interests include code vulnerability detection, software testing and explainable AI.



Zhongda Lu received the Master degree from Xi-Dian University, China, in 2003. He is currently the Technical Director of the AI Team at Beijing Baolande Software, China. His main research interests include AIOps, LLM and its application on AI Agent.



Ting He received the Bachelor's degree in Mathematics from Central South University, China, in 2013. currently the AI Algorithm Team Lead at Beijing Baolande Software, China. main research interests include AIOps, large language models (LLMs), and their application in AI Agents.



Chengwei Liu received the Bachelor's degree from Hunan University of Humanities, Science and Technology, China, in 2013. Currently serving as an AI Development Engineer at Beijing Baolande Software, China. Main research and practical interests include AIOps, AI-driven application development, and large-scale AI solutions for enterprise operations.